FAST NUCES

National University of Computer & Emerging Sciences | Peshawar Campus

AI LAB

PYTHON CRASH COURSE

For C++ Warriors Entering the AI Battlefield

Instructor: **Hamza Raziq Khan**

Program: **BS Computer Science | 6th Semester**

Duration: **6 Hours**

Mission: **Transform C++ ninjas into Python-powered AI builders**

"In C++ you had to earn every semicolon. In Python, you earn freedom."

LAB 01

Python Fundamentals: Goodbye Semicolons, Hello Colons!

Duration: 3 Hours

Learning Objectives

By the end of this lab, you will be able to:

1. Write and execute Python scripts with confidence, understanding the key differences from C++.

2. Use Python's dynamic typing, built-in data structures (lists, dictionaries, tuples, sets), and string manipulation.

3. Implement control flow (if/elif/else, for, while) and write functions with default and keyword arguments.

4. Leverage list comprehensions and basic file I/O for real-world data processing tasks.

- - -

Part A: Instructor-Led Session

Welcome to the dark side... where indentation actually matters and semicolons are optional (well, nonexistent). If you survived C++ pointers, Python will feel like a vacation.

1. The Great Translation: C++ vs Python

You already know how to think like a programmer. Python just lets you express those thoughts with less typing and fewer compiler errors. Let's see the key translations:

C++ (What You Know)	Python (Your New Best Friend)	Key Difference
`int x = 5;`	`x = 5`	No type declaration needed
`string s = "hi";`	`s = "hi"`	No std:: namespace
`cout << x << endl;`	`print(x)`	One word. Done.
`int arr[] = {1,2,3};`	`arr = [1, 2, 3]`	Lists are dynamic
`for(int i=0;i<n;i++)`	`for i in range(n):`	So much cleaner
`if(x > 5) {`	`if x > 5:`	Colons replace braces

`#include <vector>`	`import numpy`	No header files
`// comment`	`# comment`	Hash, not double-slash

C++ Veteran's Survival Tip

In Python, indentation is NOT optional. It replaces your beloved curly braces {}. Use 4 spaces (not tabs). Your code won't run if indentation is wrong. Yes, whitespace has power here.

2. Variables and Data Types

Python uses dynamic typing. You don't declare types; Python figures it out. Variables are like shape-shifters compared to C++'s strict casting.

```
# Python figures out the type automatically
name = "FAST NUCES"          # str
semester = 6                  # int
gpa = 3.75                    # float
is_passed = True              # bool (capital T/F!)
courses = ["AI", "DB", "OS"]  # list
student = {"name": "Ali", "roll": "21P-0001"}  # dict

# Type checking
print(type(gpa))     # <class 'float'>
print(isinstance(semester, int))  # True

# Type conversion (casting)
x = int("42")        # str to int
y = str(3.14)        # float to str
z = float("2.5")     # str to float
```

Mind-Blowing for C++ Devs

Variables can change type mid-program: x = 5 then x = "hello" is perfectly legal. Python doesn't care. Freedom or chaos? You decide.

3. Strings: Your New Superpower

Forget char arrays and string.h. Python strings are objects with built-in superpowers. f-strings alone will make you never want to go back to cout.

```
# f-strings (formatted string literals) - your new best friend
name = "Hamza"
course = "AI Lab"
```

```
print(f"Welcome to {course}, taught by {name}!")

# String methods (no #include needed!)
msg = "  Hello FAST NUCES  "
print(msg.strip())         # removes whitespace
print(msg.lower())         # "  hello fast nuces  "
print(msg.replace("FAST", "SPEED"))  # string replacement

# Slicing (C++ has no equivalent this clean)
text = "PYTHON"
print(text[0:3])    # "PYT" (start:end, end excluded)
print(text[::-1])   # "NOHTYP" (reversed!)
print(text[::2])    # "PTO" (every 2nd char)

# Split and Join
csv_line = "Ali,21P-0001,3.5"
parts = csv_line.split(",")     # ["Ali", "21P-0001", "3.5"]
rejoined = " | ".join(parts)    # "Ali | 21P-0001 | 3.5"
```

4. Control Flow: Same Logic, Less Typing

The logic you learned in C++ is identical. Only the syntax changes. No parentheses around conditions, colons instead of braces.

```
# if / elif / else
grade = 85
if grade >= 90:
    print("A grade - Excellent!")
elif grade >= 80:
    print("B grade - Good job!")
elif grade >= 70:
    print("C grade - Keep going!")
else:
    print("Needs improvement")

# for loops with range
for i in range(5):           # 0, 1, 2, 3, 4
    print(i)

for i in range(2, 10, 3):    # 2, 5, 8 (start, stop, step)
    print(i)

# for loop over a list (no index needed!)
courses = ["AI", "DB", "Networks"]
for course in courses:
    print(f"I am taking {course}")
```

```
# while loop
count = 0
while count < 5:
    print(count)
    count += 1   # Note: no count++ in Python!
```

5. Data Structures: Lists, Dicts, Tuples, Sets

Forget implementing your own linked list or hash map. Python gives you powerful built-in data structures that just work.

```
# ---- LISTS (like C++ vector, but better) ----
marks = [85, 92, 78, 95, 88]
marks.append(91)           # Add to end
marks.insert(0, 100)       # Insert at index 0
marks.pop()                # Remove last element
marks.sort()               # Sort in-place
print(marks[1:4])          # Slicing: [85, 88, 92]
print(len(marks))          # Length

# ---- DICTIONARIES (like C++ map, but simpler) ----
student = {
    "name": "Ali Khan",
    "roll": "21P-0042",
    "gpa": 3.6,
    "courses": ["AI", "DB"]
}
print(student["name"])           # Access by key
student["semester"] = 6          # Add new key
print(student.get("age", "N/A")) # Safe access with default

# ---- TUPLES (immutable lists) ----
coordinates = (33.99, 71.51)  # Peshawar coordinates!
# coordinates[0] = 34.0       # ERROR! Tuples can't change

# ---- SETS (unique elements, like C++ set) ----
skills = {"Python", "C++", "Python"}  # Duplicate removed
print(skills)  # {"Python", "C++"}
```

6. Functions: Clean, Flexible, Powerful

```
# Basic function
def greet(name):
    return f"Salam, {name}! Welcome to AI Lab."
```

```
# Default arguments
def calculate_grade(marks, total=100):
    percentage = (marks / total) * 100
    if percentage >= 90: return "A"
    elif percentage >= 80: return "B"
    elif percentage >= 70: return "C"
    else: return "F"

print(calculate_grade(85))        # Uses default total=100
print(calculate_grade(42, 50))    # Custom total

# Multiple return values (C++ can't do this easily!)
def get_stats(numbers):
    return min(numbers), max(numbers), sum(numbers)/len(numbers)

low, high, avg = get_stats([85, 92, 78, 95, 88])
print(f"Min: {low}, Max: {high}, Avg: {avg:.2f}")

# Lambda (anonymous) functions
square = lambda x: x ** 2
print(square(5))  # 25
```

7. List Comprehensions: One Line to Rule Them All

This is where Python starts to feel like a superpower. List comprehensions let you create and transform lists in a single, readable line.

```
# Traditional way (C++ thinking)
squares = []
for i in range(10):
    squares.append(i ** 2)

# Python way (list comprehension)
squares = [i ** 2 for i in range(10)]

# With condition (filter)
even_squares = [i ** 2 for i in range(10) if i % 2 == 0]
# Result: [0, 4, 16, 36, 64]

# Nested comprehension
matrix = [[i * j for j in range(1, 4)] for i in range(1, 4)]
# Result: [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

# Dictionary comprehension
names = ["Ali", "Sara", "Ahmed"]
name_lengths = {name: len(name) for name in names}
```

```
# Result: {"Ali": 3, "Sara": 4, "Ahmed": 5}
```

- - -

Part B: Practice Problems (130 Minutes)

Time to get your hands dirty! Each problem is a real-world scenario. Start from the top and work your way down. They get progressively harder. Ask your instructor if you're stuck for more than 10 minutes.

P1 The Chai Stall Calculator

Your university's famous chai wala (Peshawar's finest!) wants to digitize his business. He sells chai for Rs. 50, paratha for Rs. 40, and anda (egg) for Rs. 30. Write a program that takes an order as a dictionary (e.g., {"chai": 3, "paratha": 2, "anda": 1}) and prints a formatted receipt showing each item, its quantity, subtotal, and the grand total. If the total exceeds Rs. 500, apply a 10% "best customer" discount. Add a tip suggestion of 5%, 10%, and 15%.

```
# Expected output for order = {"chai": 3, "paratha": 2, "anda": 1}:
# ========== CHAI WALA RECEIPT ==========
# Chai        x3    Rs. 150
# Paratha     x2    Rs.  80
# Anda        x1    Rs.  30
# ------------------------------------
# Subtotal:         Rs. 260
# Discount (10%):   Rs.   0
# Grand Total:      Rs. 260
# ------------------------------------
# Tip Suggestions:
#   5%  -> Rs. 13.00
#  10%  -> Rs. 26.00
#  15%  -> Rs. 39.00
# ====================================
```

P2 CGPA Analyzer: Are You Dean's List Material?

Write a program that stores 5 students' data as a list of dictionaries. Each student has a name, roll number, and a list of 5 course GPAs. Calculate each student's CGPA (average of course GPAs). Then: (a) Print all students sorted by CGPA (highest first), (b) Identify who made the Dean's List (CGPA >= 3.5), (c) Find the student with the highest and lowest CGPA, (d) Calculate the class average CGPA.

```
# Sample data structure
students = [
    {"name": "Ali Khan", "roll": "21P-0001",
     "gpas": [3.7, 3.5, 3.8, 3.6, 3.9]},
    {"name": "Sara Ahmed", "roll": "21P-0002",
     "gpas": [3.9, 4.0, 3.7, 3.8, 3.9]},
    # ... add 3 more students yourself
]

# Hint: Use sum()/len() for averages
# Hint: Use sorted() with key parameter
# Hint: sorted(students, key=lambda s: ..., reverse=True)
```

P3 The Password Strength Analyzer

FAST IT department needs a password checker. Write a function check_password(password) that analyzes a password and returns a strength score (0-5) based on these rules: +1 for length >= 8, +1 for containing uppercase, +1 for containing lowercase, +1 for containing a digit, +1 for containing a special character (!@#$%^&*). Also return a list of missing criteria. Map scores to labels: 0-1 = "Weak (your password is basically 'password123')", 2-3 = "Medium (getting there...)", 4-5 = "Strong (even hackers would cry)". Test with at least 5 different passwords.

```
# Hints:
# any(c.isupper() for c in password)  # Check for uppercase
# any(c.isdigit() for c in password)   # Check for digit
# any(c in "!@#$%^&*" for c in password)  # Special chars

# Expected: check_password("abc") -> (1, ["Too short", ...])
# Expected: check_password("MyP@ss1") -> (4, [...])
```

P4 The Peshawar Food Festival Menu Randomizer

The annual Peshawar Food Festival needs a random menu generator! You have 4 categories: Mains (chapli kabab, namkeen gosht, karahi, shinwari tikka, peshawari pulao), Sides (raita, salad, naan, chutney), Drinks (doodh patti, green tea, lassi, sugarcane juice), and Desserts (firni, jalebi, kheer, gulab jamun). Write a program that: (a) Generates a random combo meal with 1 item from each category, (b) Generates a party platter with 3 mains, 2 sides, 2 drinks, 1 dessert (no repeats!), (c) Calculates random prices (Mains: 300-800, Sides: 50-200, Drinks: 50-150, Desserts: 100-250) and shows total.

```
# Hint: import random
# random.choice(list)     # Pick one random item
# random.sample(list, k)  # Pick k unique items
# random.randint(a, b)    # Random integer [a, b]
```

P5 Student Attendance Tracker

Build an attendance tracker that stores data as a dictionary where keys are student names and values are lists of "P" (present) or "A" (absent) for each week. Pre-populate with 5 students and 10 weeks of random attendance. Then: (a) Calculate attendance percentage for each student, (b) Flag students below 75% (short attendance warning!), (c) Find the week with the lowest overall attendance, (d) Print a formatted attendance report that looks professional. Bonus: Add a function that predicts if a student will fall below 75% by end of semester given current trend.

```
# Sample structure
attendance = {
    "Ali":   ["P","P","A","P","P","A","P","P","P","A"],
    "Sara":  ["P","P","P","P","P","P","P","P","P","P"],
    # ...
}
# Hint: count() method -> ["P","A","P"].count("P") returns 2
```

P6 The Number Guessing Game (If You Finish Early)

Build a number guessing game where the computer picks a random number between 1 and 100. The player gets 7 attempts. After each guess, tell them "Too high!", "Too low!", or "Correct!". Track the number of attempts. After the game, show stats: attempts used, efficiency percentage (fewer = better). Add a difficulty system: Easy (1-50, 10 tries), Medium (1-100, 7 tries), Hard (1-500, 9 tries). Bonus: Keep a high score list of the top 5 games.

- - -

Part C: Lab Tasks (Graded Homework)

Complete the following tasks at home and submit by the next lab session. Each solution must be a single .py file, well-commented, and should produce the exact output format described and each file should be present in a main zip file. Plagiarism will result in zero marks for all involved parties.

T1

The FAST NUCES Transcript Generator [25 marks]

Create a program that generates a formatted academic transcript. Define a student with: name, roll number, department, semester, and a list of 6 courses. Each course has: name, credit hours (1-4), and marks obtained (0-100). Your program must: (a) Convert marks to letter grades using FAST's grading policy (A: 90+, A-: 85+, B+: 80+, B: 75+, B-: 70+, C+: 65+, C: 60+, D: 50+, F: <50), (b) Calculate GPA for each course (A=4.0, A-=3.67, B+=3.33, etc.),

(c) Calculate semester GPA weighted by credit hours, (d) Print a beautifully formatted transcript that would make the registrar proud. Use f-strings for alignment.

T2

The Ultimate Word Frequency Analyzer [25 marks]

Write a program that analyzes any given text paragraph (hardcode a paragraph of at least 100 words about any topic). Your analyzer must: (a) Count total words, sentences, and unique words, (b) Find the top 10 most frequent words (excluding common stop words: 'the', 'a', 'an', 'is', 'in', 'to', 'and', 'of', 'for', 'it', 'on', 'that', 'with', 'was'), (c) Find the longest and shortest words, (d) Calculate average word length, (e) Create a simple text-based bar chart showing word frequencies using '#' characters. All processing must use Python string methods and dictionaries (no external libraries).

T3

The Cricket Match Simulator [25 marks]

Simulate a T20 cricket match between two teams (Peshawar Zalmi vs Islamabad United, obviously). For each ball: generate a random outcome from [0, 1, 2, 3, 4, 6, 'W'] with realistic probabilities (0: 30%, 1: 25%, 2: 15%, 3: 5%, 4: 12%, 6: 8%, W: 5%). Each team has 10 wickets and 120 balls. Track: runs per over, total score, wickets, run rate, and individual batsman scores (just number them Batsman 1-11). After both innings, print a full scorecard showing: each batsman's score and balls faced, bowling figures per over, fall of wickets, team totals, run rates, and who won (with margin). Use functions to keep code modular.

T4

The Matrix Operations Calculator (C++ Brain, Python Hands) [25 marks]

Build a matrix calculator that works with 2D lists (no NumPy!). Implement: (a) Matrix addition and subtraction, (b) Matrix multiplication (the real one, not element-wise), (c) Matrix transpose, (d) Matrix determinant (for 2x2 and 3x3), (e) Scalar multiplication. Each operation should be a separate function. Include input validation (check dimensions match for operations). Create a menu-driven program where the user can input matrices and choose operations. Test with at least three matrix pairs of different sizes. Print matrices in a nicely formatted grid. This task bridges your C++ algorithmic thinking with Python syntax.

SUBMISSION GUIDELINES

1. File naming: RollNumber_Lab1_TaskNumber.py (e.g., 21P-0042_Lab1_T1.py)
2. Each file must be placed in a **zip** file and **zip** file should be named as your_roll_number_yourname.zip
3. **Plagiarism = 0 for ALL parties.**

LAB 02

Advanced Python: The AI Toolkit Awakens

Duration: 3 Hours | Teaching: 60 min | Practice: 120 min

Learning Objectives

By the end of this lab, you will be able to:

1. Implement Object-Oriented Programming in Python, mapping your C++ OOP knowledge seamlessly.
2. Use error handling, file I/O, and modules to write robust, production-quality Python code.
3. Master NumPy arrays and operations essential for AI/ML data processing.
4. Apply functional programming concepts (map, filter, reduce, lambda) for data transformations.

- - -

Part A: Instructor-Led Session (60 Minutes)

Last lab, you survived the Python basics. Today, we level up. Think of Lab 1 as learning to walk. Today, you learn to fly... and then land safely on an AI project.

1. Object-Oriented Programming: C++ Classes, Python Style

You already understand OOP from C++. The concepts are identical: classes, objects, inheritance, polymorphism. Python just removes the boilerplate. No header files, no access specifiers (well, sort of), no semicolons.

```
# C++ class -> Python class translation
class Student:
    # Constructor (__init__ = C++ constructor)
    def __init__(self, name, roll, gpa=0.0):
        self.name = name       # public by default
        self.roll = roll
        self._gpa = gpa        # _ prefix = "private" (convention)

    # Method (self = C++ this pointer, but explicit)
    def display(self):
        print(f"Student: {self.name} ({self.roll}) - GPA: {self._gpa}")
```

```
    # Property (like C++ getter/setter but Pythonic)
    @property
    def gpa(self):
        return self._gpa

    @gpa.setter
    def gpa(self, value):
        if 0 <= value <= 4.0:
            self._gpa = value
        else:
            raise ValueError("GPA must be between 0 and 4.0")

    # String representation (like C++ operator<<)
    def __str__(self):
        return f"{self.name} ({self.roll})"

    # Comparison (like C++ operator<)
    def __lt__(self, other):
        return self._gpa < other._gpa

# Creating objects
s1 = Student("Ali", "21P-0001", 3.8)
s1.display()
print(s1)    # Uses __str__
```

2. Inheritance and Polymorphism

```
# Inheritance (same concept as C++, cleaner syntax)
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        return f"I am {self.name}, age {self.age}"

class Student(Person):  # Student inherits from Person
    def __init__(self, name, age, roll):
        super().__init__(name, age)  # Call parent constructor
        self.roll = roll

    def introduce(self):  # Override (polymorphism)
        return f"{super().introduce()}, Roll: {self.roll}"

# Multiple inheritance (C++ supports it, Python makes it easier)
class TeachingAssistant(Student):
```

```
    def __init__(self, name, age, roll, course):
        super().__init__(name, age, roll)
        self.course = course

# isinstance() and issubclass()
ta = TeachingAssistant("Ahmed", 23, "19P-0005", "AI Lab")
print(isinstance(ta, Person))      # True
print(isinstance(ta, Student))     # True
```

3. Error Handling: try/except (C++ try/catch)

Same concept as C++ exceptions, different keywords. Python encourages "ask forgiveness, not permission" (EAFP) style.

```
# Basic exception handling
try:
    number = int(input("Enter a number: "))
    result = 100 / number
    print(f"Result: {result}")
except ValueError:
    print("That's not a valid number!")
except ZeroDivisionError:
    print("Can't divide by zero!")
except Exception as e:
    print(f"Something went wrong: {e}")
finally:
    print("This always executes (like C++ destructor behavior)")

# Custom exception (like C++ custom exception class)
class InvalidGPAError(Exception):
    def __init__(self, gpa):
        self.gpa = gpa
        super().__init__(f"Invalid GPA: {gpa}. Must be 0.0-4.0")

# Raising exceptions
def set_gpa(gpa):
    if not 0 <= gpa <= 4.0:
        raise InvalidGPAError(gpa)
    return gpa
```

4. File I/O: Reading and Writing Files

AI is all about data. You need to read CSVs, process text files, and save results. Python makes file I/O trivially easy.

```
# Writing to a file
with open("students.txt", "w") as f:
    f.write("Name, Roll, GPA\n")
    f.write("Ali, 21P-0001, 3.8\n")
    f.write("Sara, 21P-0002, 3.9\n")
# File auto-closes! (like RAII in C++)

# Reading from a file
with open("students.txt", "r") as f:
    header = f.readline()         # Read first line
    for line in f:                # Iterate remaining lines
        parts = line.strip().split(", ")
        print(f"Student: {parts[0]}, GPA: {parts[2]}")

# Reading entire file at once
with open("students.txt", "r") as f:
    content = f.read()            # Entire file as string
    lines = f.readlines()         # List of lines

# CSV-style processing
import csv
with open("data.csv", "r") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["name"], row["gpa"])
```

5. NumPy: The Foundation of AI in Python

NumPy (Numerical Python) is THE library that makes Python viable for AI. It provides fast array operations that would take hundreds of lines in C++. Every ML/AI library is built on top of NumPy.

```
import numpy as np

# Creating arrays
a = np.array([1, 2, 3, 4, 5])          # 1D array
b = np.array([[1, 2, 3], [4, 5, 6]])    # 2D array (matrix)

# Special arrays
zeros = np.zeros((3, 3))         # 3x3 matrix of zeros
ones = np.ones((2, 4))           # 2x4 matrix of ones
identity = np.eye(3)             # 3x3 identity matrix
rand = np.random.rand(3, 3)      # 3x3 random matrix
rng = np.arange(0, 10, 0.5)      # [0, 0.5, 1.0, ..., 9.5]

# Array operations (element-wise by default!)
x = np.array([1, 2, 3])
```

```
y = np.array([4, 5, 6])
print(x + y)          # [5, 7, 9]  (no loops needed!)
print(x * y)          # [4, 10, 18]
print(x ** 2)         # [1, 4, 9]
print(np.dot(x, y))   # 32 (dot product)

# Matrix operations
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print(A @ B)              # Matrix multiplication
print(np.linalg.inv(A))   # Matrix inverse
print(np.linalg.det(A))   # Determinant
print(A.T)                # Transpose

# Statistical operations
data = np.array([85, 92, 78, 95, 88, 76, 91])
print(f"Mean: {np.mean(data):.2f}")
print(f"Std Dev: {np.std(data):.2f}")
print(f"Max: {np.max(data)}, Min: {np.min(data)}")

# Boolean indexing (filtering)
passed = data[data >= 80]  # Only scores >= 80
print(passed)  # [85, 92, 95, 88, 91]
```

Why NumPy Matters for AI

In AI, you work with vectors, matrices, and tensors constantly. Neural networks are literally matrix multiplications chained together. NumPy makes these operations 50-100x faster than Python lists because it uses optimized C code under the hood. Think of it as having C++ speed with Python convenience.

6. Functional Programming: map, filter, reduce

These functions are heavily used in data preprocessing for AI pipelines. They let you transform data without writing explicit loops.

```
# map: apply a function to every element
temps_celsius = [0, 20, 37, 100]
temps_fahrenheit = list(map(lambda c: (c * 9/5) + 32, temps_celsius))
# Result: [32.0, 68.0, 98.6, 212.0]

# filter: keep elements that satisfy a condition
scores = [45, 67, 82, 91, 55, 73, 88]
passed = list(filter(lambda s: s >= 60, scores))
# Result: [67, 82, 91, 73, 88]

# reduce: accumulate values (like fold in C++)
from functools import reduce
```

```
numbers = [1, 2, 3, 4, 5]
product = reduce(lambda a, b: a * b, numbers)
# Result: 120 (1*2*3*4*5)

# Chaining: clean a dataset in one pipeline
raw_data = ["  Ali ", "SARA", " ahmed  ", "", "Zain"]
clean = list(filter(None, map(lambda s: s.strip().title(), raw_data)))
# Result: ["Ali", "Sara", "Ahmed", "Zain"]
```

- - -

Part B: Practice Problems (120 Minutes)

Now we combine everything: OOP, NumPy, file I/O, and more. These problems are designed to simulate real AI-adjacent tasks you'll encounter in upcoming labs.

P1 The University Course Management System (OOP)

Build a mini course management system using OOP. Create classes: (a) Person (name, age), (b) Student(Person) with roll_number, courses list, and methods to enroll/drop courses, (c) Instructor(Person) with employee_id and courses_teaching, (d) Course with code, name, instructor, enrolled_students list, and max_capacity (30). Implement: enroll student (check capacity), drop student, display roster, calculate average GPA. Create at least 3 courses, 1 instructor, and 5 students. Demonstrate enrollment, dropping, and try enrolling in a full course (handle the error gracefully).

P2 The Grade Distribution Analyzer with NumPy

Generate random exam scores for 100 students across 5 subjects using NumPy (scores between 40-100). Then: (a) Calculate mean, median, std deviation for each subject, (b) Find how many students passed each subject (>= 50) and failed, (c) Find the overall topper (highest total), (d) Create grade distribution: count how many A, B, C, D, F grades in each subject, (e) Find the correlation between any two subjects using np.corrcoef (do students who score high in one tend to score high in another?). All operations must use NumPy (no Python loops for calculations!).

```
import numpy as np
np.random.seed(42)  # For reproducibility
scores = np.random.randint(40, 101, size=(100, 5))
subjects = ["Math", "Physics", "Programming", "AI", "English"]
# Now analyze...
```

P3 The Peshawari Restaurant Review Analyzer

You have a list of 15 restaurant reviews (hardcode them as strings). Using string methods and dictionaries: (a) Count positive words (delicious, amazing, excellent, great, best, fantastic, wonderful, love) and negative words (terrible, awful, bad, worst, disgusting, horrible, poor, hate) in each review, (b) Assign a sentiment score (-1 to +1) based on positive vs negative word ratio, (c) Classify each review as Positive/Negative/Neutral, (d) Find the most common positive and negative words across all reviews, (e) Write results to a file called 'review_analysis.txt' with a formatted summary table.

P4 The Student Database File Manager

Create a complete student database using file I/O: (a) Write a function to create a CSV file with 20 students (name, roll, department, semester, CGPA), (b) Write a function to read and display all students from the file, (c) Write a function to search by roll number, (d) Write a function to update a student's CGPA, (e) Write a function to delete a student record, (f) Write a function to generate a summary report (total students, average CGPA, department-wise count, students on probation with CGPA < 2.0). All operations must read/write to the CSV file. Create a menu-driven interface.

P5 Neural Network Math: The Foundation (NumPy Challenge)

Implement the basic mathematical operations that power neural networks using only NumPy: (a) Implement sigmoid function: sigma(x) = 1/(1 + e^(-x)), (b) Implement ReLU: max(0, x), (c) Create a function that does a single neural network forward pass: output = sigmoid(weights . inputs + bias), where weights is a 3x2 matrix, inputs is a 2x1 vector, bias is a 3x1 vector, (d) Implement softmax function for a vector, (e) Calculate Mean Squared Error between predicted and actual values. Test each function with sample data and verify results make mathematical sense. This is literally what you'll build on in future AI labs!

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def relu(x):
    return np.maximum(0, x)

# Test: sigmoid(0) should be 0.5
# Test: relu(-3) should be 0, relu(5) should be 5
# Forward pass: try weights=[[0.1,0.2],[0.3,0.4],[0.5,0.6]],
#               inputs=[1.0, 0.5], bias=[0.1, 0.2, 0.3]
```

- - -

Part C: Lab Tasks (Graded Homework)

These tasks integrate everything from both labs. They are intentionally challenging to ensure you're truly ready for the AI labs ahead. Start early.

T1	**The Complete Library Management System (OOP + File I/O)** **[25 marks]** Build a Library Management System using OOP and file persistence. Classes: Book (isbn, title, author, quantity, available_quantity), Member (id, name, borrowed_books list, max_books=3), and Library (name, books dict, members dict). Implement: (a) Add/remove books, (b) Register/remove members, (c) Borrow book (check availability, check member limit, update quantities), (d) Return book (update records), (e) Search books by title or author (partial match), (f) Display all overdue simulation (randomly assign borrow dates, flag books borrowed > 14 days ago), (g) Save/Load entire library state to/from a JSON file using Python's json module. Include proper error handling throughout. Create a menu-driven interface.
T2	**The Complete Data Analysis Pipeline (NumPy + File I/O)** **[25 marks]** Create a data analysis pipeline: (a) Generate a synthetic dataset of 200 students with columns: StudentID, Age (18-25), AttendancePercentage (40-100), StudyHoursPerWeek (0-40), MidtermScore (0-100), FinalScore (0-100) using NumPy random. Save to CSV. (b) Load the CSV and compute: mean, median, std, min, max for each numeric column. (c) Find correlation matrix between all numeric columns. (d) Implement a simple linear predictor: predict FinalScore from MidtermScore using the formula: predicted = mean(Final) + correlation * std(Final)/std(Mid) * (midterm - mean(Mid)). (e) Calculate the Mean Absolute Error of your predictions. (f) Write a comprehensive analysis report to a text file. All numerical operations must use NumPy.
T3	**The AI Chatbot Framework (OOP + Functional Programming)** **[25 marks]** Build a rule-based chatbot framework using OOP: Create a Chatbot class with: name, response_rules (dict mapping keywords to responses), conversation_history (list), and sentiment_tracker. Implement: (a) A respond() method that checks user input against keyword rules and returns the best match (check for multiple keyword matches and pick the most specific), (b) Add at least 30 rules covering: greetings, farewells, questions about FAST, questions about Peshawar, questions about AI, jokes, compliments, and insults (respond gracefully!), (c) Track conversation sentiment using positive/negative word counting, (d) If sentiment drops too low, the bot should try to cheer the user up, (e) Save conversation history to a file, (f) Add a learning mode: if the bot doesn't know an answer, ask the user and add the new rule. Use lambda functions and map/filter where appropriate for text processing.

T4

The Mini NumPy (Implement from Scratch!) [25 marks]

This is the boss fight. Implement a MiniNumPy class that works on 2D lists (no importing NumPy). It should support: (a) Creating arrays from lists, zeros, ones, and random, (b) Shape and reshape, (c) Element-wise addition, subtraction, multiplication, (d) Matrix multiplication (@ operator via __matmul__), (e) Transpose, (f) Sum across rows, columns, or entire matrix, (g) Mean and standard deviation, (h) Boolean indexing (elements > threshold), (i) Pretty-print the matrix with aligned columns. Implement using operator overloading (__add__, __mul__, __matmul__, __str__, __getitem__). Test by solving a system of equations or computing the forward pass of a neural network layer using YOUR class. This task proves you truly understand both Python and the math behind AI.

SUBMISSION GUIDELINES

1. File naming: RollNumber_Lab1_TaskNumber.py (e.g., 21P-0042_Lab1_T1.py)
2. Each file must be placed in a **zip** file and **zip** file should be named as your_roll_number_yourname.zip
3. **Plagiarism = 0 for ALL parties.**

You've Completed the Python Crash Course!

From Lab 3 onward, we build actual AI systems.

Quick Reference Cheat Sheet

Python Docs: docs.python.org | NumPy Docs: numpy.org/doc
Practice: leetcode.com/python | hackerrank.com/python

"The best way to learn Python is to write Python. So stop reading and start coding!"

Prepared by Hamza Raziq Khan | FAST NUCES Peshawar | AI Lab 2026